

OPTIMIZATION-FIRST LLM AGENTS

NemobotIR

A research path for turning agent workflows from hand-written prototypes into analyzable, optimizable processes.

WORKFLOW OBJECTIVE

more valid tool use

**fewer
hallucinations**

lower token cost

**less privacy
exposure**

Flexible prototyping, but not yet an optimization platform

Nemobot already builds tool-using chatbot workflows. The research opportunity is to make the workflow itself a formal object we can analyze before deployment.

TODAY

Manual scripts

Prompts, tools, and JavaScript logic are flexible but hard to compare systematically.

MISSING

Canonical process model

There is no standard workflow representation for states, transitions, tools, and tests.

NEEDED

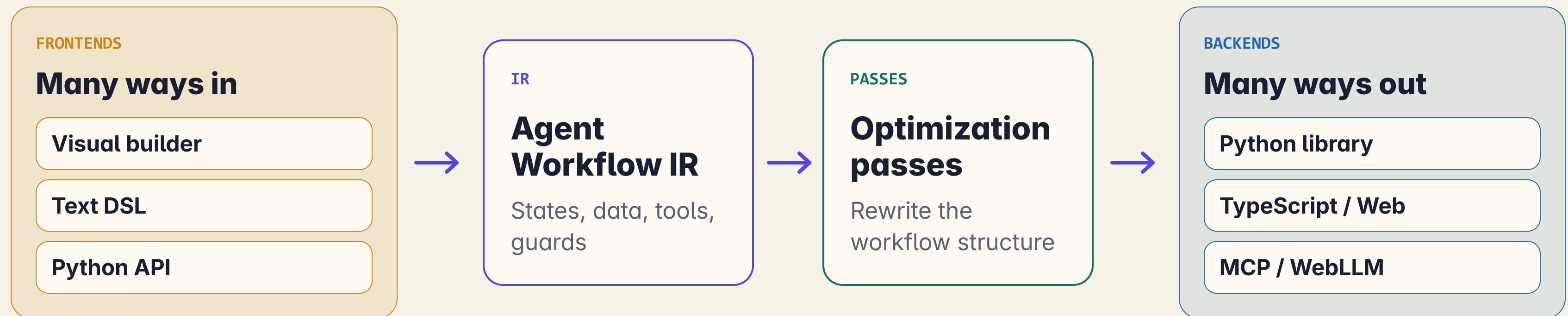
Optimization layer

We need static checks and transformations before the agent runs in the real world.

KEY TURN Make the workflow explicit, so we can analyse and optimize it before deployment

Compile the workflow, not the model

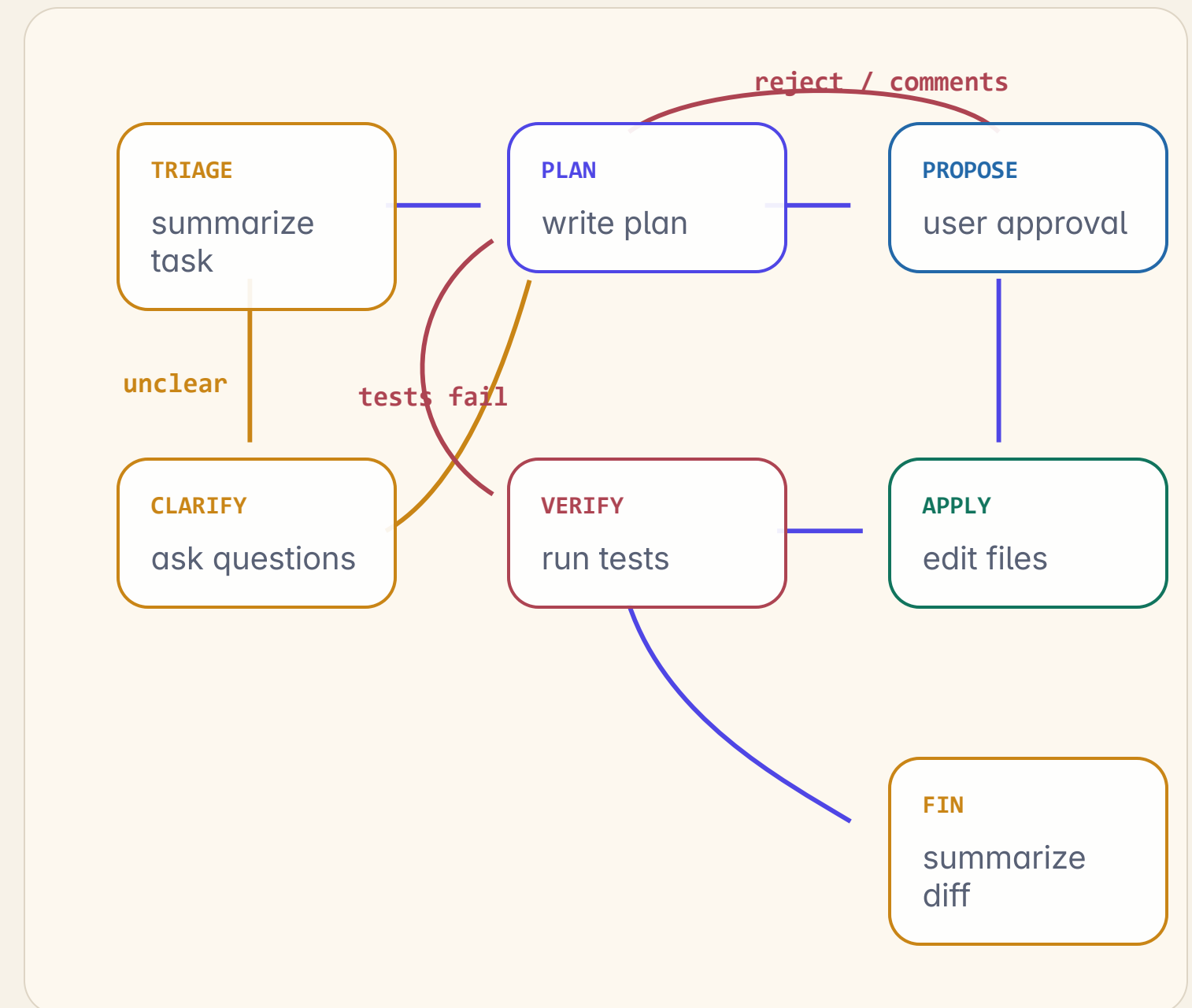
Different authoring interfaces lower into the same Agent Workflow IR. Optimization happens once in the middle, then code generation targets the deployment runtime.



MAKE BEHAVIOR EXPLICIT

A workflow becomes a state machine we can reason about

Each stage has typed inputs, required capabilities, and explicit transitions for approval and repair loops.



RESEARCH FRAMING

The research object is the workflow design space

Different decompositions, tool sets, prompts, and model choices become candidate solutions.

CONCEPTUAL OBJECTIVE

maximize success(workflow)

- $w1 * \text{token_cost}$
- $w2 * \text{latency}$
- $w3 * \text{risk}$

-
- valid tool schemas
 - policy constraints
 - supported factual claims
 - human approval for risky actions

Optimization passes are workflow transformations

Instead of optimizing memory layout, these passes optimize reliability, cost, privacy, and tool-use validity.

CONTEXT MINIMIZATION

Send each state only live variables.

Less token cost, less privacy exposure.

TOOL VISIBILITY

Expose only tools feasible in this state.

Fewer invalid or irrelevant tool calls.

VERIFIER INSERTION

Add checks after risky operations.

Supported claims and safer actions.

STATE SPLIT

Break vague steps into smaller ones.

More reliable behavior for small models.

STATE FUSION

Merge unnecessary adjacent states.

Lower latency when separation is wasteful.

MODEL ROUTING

Assign cheap, strong, or local models by state.

Resource allocation over the workflow.

MAKE THE CLAIM TESTABLE

Compare optimized workflows against strong baselines

The key evidence is not that the DSL exists. It is that optimization improves measurable agent behavior.

Free-form ReAct agent

Natural baseline: flexible but unstable and hard to audit.

Hand-written state machine

Separates structure from chat, but no compiler optimization.

NemobotIR unoptimized

Same workflow representation, no optimization passes applied.

NemobotIR optimized

Passes enabled: context, tools, verifier, routing, split or fusion.

MEASURE

task success, invalid tool calls, policy violations, latency, token cost, trace reproducibility

LLVM: one intermediate language, many targets

LLVM lets many source languages compile to many hardware targets through one shared intermediate representation.

C SOURCE

```
int add(int a, int b) {  
    return a + b;  
}
```

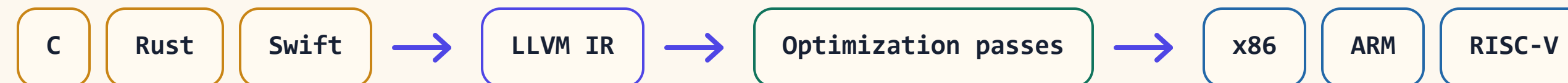
LLVM IR

```
define i32 @add(i32 %a, i32 %b) {  
    %result = add i32 %a, %b  
    ret i32 %result  
}
```

x86 ASSEMBLY

```
add:  
    mov    eax, edi  
    add    eax, esi  
    ret
```

PIPELINE



BACKGROUND: MULTI-LEVEL COMPILERS

MLIR: multiple levels of representation

MLIR extends LLVM with a chain of dialects. Each level has its own optimization passes before lowering further.

TENSORFLOW MODEL

```
y = tf.matmul(A, x) + b
```



MLIR 'TENSOR' DIALECT

```
%0 = linalg.matmul  
  ins(%A, %x) outs(%init)
```



MLIR 'LINALG' DIALECT

```
scf.for %i = 0 to %N {  
  scf.for %j = 0 to %M { ... }  
}
```



MLIR 'LLVM' DIALECT -> GPU KERNELS

```
nvvm.ld.global.f32 ...  
nvvm.fma.ftz.f32 ...
```

SAME PATTERN, DIFFERENT OBJECT

LLVM, MLIR, and NemobotIR share the compiler philosophy

All three define a structured representation, run optimization passes, and lower to executable backends. The difference is what they compile.

	LLVM	MLIR	NemobotIR
Source	C, Rust, Swift	MLIR dialect frontend	Agent workflow DSL
IR	SSA values, basic blocks	Multiple dialects	States, transitions, tools
Passes optimize	Speed, memory	Domain-specific + lowering	Reliability, cost, privacy
Backend	x86, ARM, RISC-V	LLVM IR	Python, TypeScript, WebLLM, MCP

Two ways Moreau fits into NemobotIR

Moreau compiles convex optimization problems into fast GPU-batched solvers. It connects to NemobotIR at two levels.

ANGLE 1: TOOL NODE

Moreau as a callable tool inside agent workflows

The compiled agent decides when to invoke a convex solve. Moreau executes it on GPU. The workflow controls the decision; Moreau supplies the math.

```
tool solve_portfolio(P, q) -> weights[]  
  backend: moreau
```

ANGLE 2: OPTIMIZATION SOLVER

Moreau as the engine inside NemobotIR optimization passes

Some workflow optimization passes can be cast as mathematical programs. Moreau solves them at compile time.

```
model routing: minimize cost  
  subject to quality >= threshold
```

TVM: compiler for ML execution

TVM takes ML models and tensor programs, optimizes how they run on hardware, then emits deployable runtime modules.

WHAT TVM COMPILES

ML computation into fast hardware code

PyTorch / ONNX

- > Relax graph IR
- > TensorIR kernels
- > LLVM / CUDA
- > runtime.Module

SAME SHAPE

IR + passes + lowering + runtime

TVM: optimizes tensor graphs and kernels.

NemobotIR: optimizes agent workflow behavior.

They operate at different layers of the stack.

INTEGRATION POINT

TVM can power local model states

NemobotIR can decide that a state should run locally; TVM compiles that classifier, embedder, reranker, or SLM to the target device.

```
State ClassifyIntent
  backend: tvm
  target: wasm / cuda
```

One IR, many ways in, many ways out

The same optimization model can be written in AMPL, CVX, or on paper, and solved with Gurobi, MOSEK, or a custom algorithm. NemobotIR separates the workflow definition from the *execution*. The IR stays the same. Everything above and below is interchangeable.

FRONTENDS

Define the workflow your way

Text DSL Quick scripting, version-controllable

Visual builder Non-programmer domain experts

Python API Programmatic agent construction

JSON / YAML config CI pipelines, declarative config

BACKENDS

Run the workflow where you need it

Python library Server-side, integrates with ML stack

TypeScript / Web Browser-native, zero-server privacy

MCP protocol Interop with any agent framework

WebLLM / local Local models, no cloud dependency

KEY INSIGHT Add a new frontend — every backend benefits. Add a new backend — every frontend benefits.